	MMCS	ML2R
TMS_{5-2}	0.998 ± 0.000	0.893 ± 0.004
TMS_{40-10}	0.996 ± 0.003	0.935 ± 0.001

Table 1: Mean max cosine similarity (MMCS) and mean L2 ratio (ML2R) with their standard deviations (to 3 decimal places) between learned parameter components and target model weights for TMS. The MMCS is very close to 1.0, indicating that every column in the target model has a corresponding column in one of the components that points in almost the same direction. The ML2R is below 1.0, indicating some amount of shrinkage in the components compared to the original model.

It is worth reflecting on the differences between the APD solution and the decompositions that other commonly used matrix decomposition methods would yield, such as singular value decomposition [Millidge and Black, 2022, Meller and Berkouk, 2023] or non-negative matrix factorization [Petrov et al., 2021, Voss et al., 2021]. Those methods can find at most $\text{rank}(W) = \min(m_1, m_2)$ components, and therefore could not decompose W into its ground truth mechanisms even in principle.

The toy model studied in this section was initially developed in order to demonstrate that neural networks can represent variables 'in superposition' using an overcomplete basis of the bottleneck activation space. However, our work decomposes the model, not activation space. Nevertheless, the mechanisms identified by our method *imply* an overcomplete basis in the activation space: The rank 1 mechanisms $Z^{(c)}$ can be expressed as an outer product of their (un-normed) left and right singular vectors $W_{:,c}e_c^\top$. The left singular vectors (corresponding to the columns of W) are an overcomplete basis of the m_1-dimensional hidden activation space[6]. Parameter vectors can thus imply overcomplete bases for the activation spaces that they interact with, even though they do not form an overcomplete basis for parameter space.

The structure of this matrix decomposition is also revealing: We can think of P_c as 'reading' from the $e_c^\top$ direction in the input space and projecting to the $W_{:,c}$ direction in the bottleneck activation space (Figure 4). Since we use W and $W^\top$ in this model, in the next layer we can also think of this parameter component 'reading' from the $W_{:,c}^\top$ in the bottleneck activation space and projecting to the e_c direction in the pre-ReLU activation space. In the backward pass, the roles are reversed: Directions that were 'reading' directions for activations become 'projecting' directions for gradients, and vice versa. In general, networks will learn parameter components consisting of matrices whose right singular vectors align with the hidden activations on the forward pass and whose left singular vectors align with gradients of the output with respect to the preactivations on the backward pass. Thus, they will ignore directions along which there are no activations, as well as directions that have no downstream causal effects.

3.2 Toy Model of Compressed Computation

While the previous example (TMS) analyzed APD on a model that stored more features than dimensions, here we examine APD on a model performing more computations than it has neurons — a phenomenon that we term *compressed computation*. We chose this model because neural networks trained on realistic tasks may often perform more computations than they have neurons. Compressed computation is very similar to the "Computation in Superposition" toy model introduced by Elhage et al. [2022], but our architecture and task differ. A key characteristic of representation in superposition [Elhage et al., 2022] and computation in superposition [Bushnaq and Mendel, 2024] is a dependence on input sparsity. We suspect our model's solutions to this task might not depend on the sparsity of inputs as much as would be expected, potentially making 'compressed computation' and 'computation in superposition' subtly distinct phenomena. But we could not conclusively establish that distinction, since experiments investigating it transpired to be more complicated than they initially appeared. We leave a more detailed study of this distinction for future work. To avoid potential confusion, we opted for a distinct term.

We train a target network to approximate a function of sparsely activating inputs $x_i \in [-1, 1]$, using a Mean Squared Error (MSE) loss between the model output and the labels. The labels we train the

[6]Equivalently, since we use W and $W^\top$ for the matrices in this model, the right hand components of $e_c W_{:,c}^\top$ also imply an overcomplete basis of the m_1-dimensional hidden activation space.

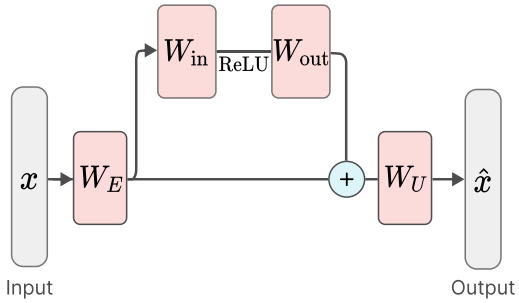

Figure 5: The architecture of our Toy Model of Compressed Computation using a 1-layer residual MLP. We fix W_E to be a randomly generated matrix with unit norm rows, and $W_U = {W_E}^\top$.

model to predict are produced by the function $y_i = x_i + \text{ReLU}(x_i)$. Crucially, the task involves learning to compute more ReLU functions than the network has neurons.

The target network is a residual MLP, consisting of a residual stream width of $d_{\text{resid}} = 1000$, a single MLP layer of width $d_{\text{mlp}} = 50$, a fixed, random embedding matrix with unit norm rows W_E, an unembedding matrix $W_U = W_E^\top$, and 100 input features. See Figure 5 for an illustration of the network architecture. The large residual stream $d_{\text{resid}} = 1000$ was chosen as the trained target network performed better than the naive monosemantic baseline in this setting (small values of d_{resid} lead to higher interference and thus worse model performance). We chose fixed, instead of trained, embedding matrices to make it simpler to calculate the optimal monosemantic baseline and to simplify training.

A naive solution to this task is to dedicate one neuron each to the computation of the first d_{mlp} functions, and to ignore the rest. This monosemantic baseline solution would perform perfectly for inputs that contained active features in only the first d_{mlp} input feature indices but poorly for all other inputs.

To understand how each neuron participates in computing the output for a given input feature, we measure what we call the neuron's *contribution* to each input feature computation. For each neuron, this contribution is calculated by multiplying two terms:

1. How strongly the neuron reads from input feature i (given by $W_{\text{IN}} W_{E[:,i]}$).
2. How strongly the neuron's output influences the model's output for index i (given by $W_{U[i,:]} W_{\text{OUT}}$).

Mathematically, we compute neuron contributions for each input feature computation $i \in [0, 99]$ by $(W_{U[i,:]} W_{\text{OUT}}) \odot (W_{\text{IN}} W_{E[:,i]})$, where $\odot$ denotes element-wise multiplication. A large positive contribution indicates that the neuron plays an important role in computing the output for input feature i. Figure 6 (top) shows the neurons involved in the computation of the first 10 input features of the target model and their corresponding contribution values. We analyze this target model in more detail in Appendix C.1.

The goal for APD in this setting is to learn parameter components that correspond to the computation of each input feature in the target model, despite these computations involving neurons that are used to compute multiple input features. For simplicity, we only decompose the MLP weights and do not decompose the target model's embedding matrix, unembedding matrix, or biases.

We found that parameter components often 'die' during training, such that no input from the training dataset can activate them. For this reason, we train with 130 parameter components. This gives APD a better chance of learning all 100 of the desired parameter components corresponding to unique input feature computations. More APD training details are given in Appendix D.

APD Results: Toy Model of Compressed Computation

Despite the target model computing more functions (100) than it has neurons (50), we find that APD can indeed learn parameter components that each implement $y_i = x_i + \text{ReLU}(x_i)$ for unique input dimensions $i \in \{0, \cdots, 99\}$. Figure 6 provides a visual representation of a set of learned parameter

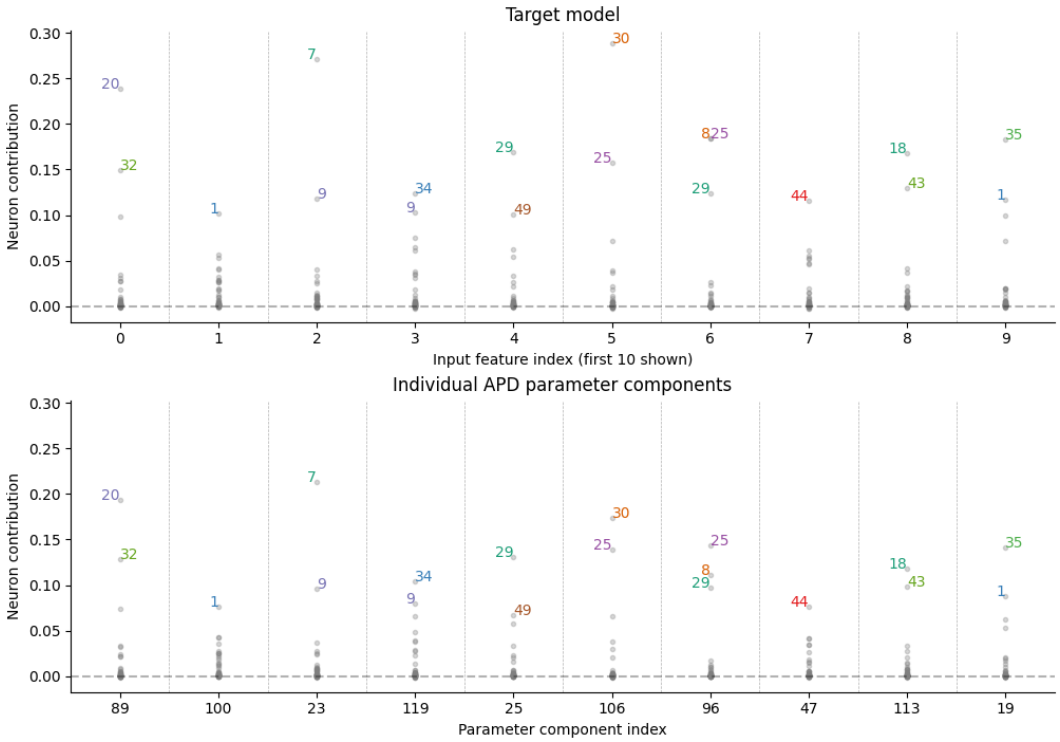

Figure 6: Similarity between target model weights and APD model components for the first 10 (out of 100) input feature dimensions. **Top**: Neuron contributions measured by $(W_{U[i,:]}W_{\text{OUT}}) \odot (W_{\text{IN}}W_{E[:,i]})$ for each input feature index $i \in [0, 9]$, where $\odot$ is an element-wise product. **Bottom**: Neuron contributions for the predominant parameter components, measured by $\max_k[(W_{U[i,:]}W_{\text{OUT}k}) \odot (W_{\text{IN}k}W_{E[:,i]})]$ for each feature index $i \in [0, 9]$. The neurons are numbered from 0 to 49 based on their raw position in the MLP layer. An extended version of this figure showing all input features and parameter components can be found here.

components. It shows how the computation that occurs for each input feature in the target network (top) is well replicated by individual parameter components in the APD model (bottom). We see that, for each input feature, there is a corresponding parameter component that uses the same neurons to compute the function as the target model does. Note that while we do not see a perfect match between the target model and the APD model, a perfect match would not actually be expected nor desirable: the neuron contribution scores of the target model can contain interference terms from the overlapping mechanisms of other features, which a single APD parameter component is likely to filter out. However, there is some 'shrinkage', similar to what we observe in the results on the TMS model (see Section 3.1). Here, much of the shrinkage is due to batch top-k forcing APD on some batches to activate more components than there are features in the input, thereby spreading out input feature computations across multiple components. We discuss some of the trade-offs when setting batch top-k in Appendix C.2.

Next, we investigate whether individual APD components have minimal influence on forward passes where their corresponding input feature is not active using a Causal Scrubbing-inspired experiment [Chan et al., 2022]: When performing a forward pass we ablate half of the APD model's parameter components, excluding the ones that correspond to the currently active inputs (the 'scrubbed' run). We compare this to ablating half of the parameter components *including* those that correspond to currently active inputs ('anti-scrubbed'). Figure 7 gives a visual illustration of the output of multiple 'scrubbed' and 'anti-scrubbed' runs for a one-hot input $x_{42} = 1$. We see that ablating unrelated components perturbs the output only slightly, barely affecting the overall shape.

To investigate whether this holds true for all components and on the training data distribution, we collect MSE losses into a histogram (Figure 8). We find that the 'scrubbed' runs (i.e. ablating unrelated parameter components – pink histogram) does not cause a large increase in the MSE loss with respect to target network outputs. On the other hand, the anti-scrubbed runs (i.e. ablating parameter components that are deemed to be responsible for the computation – green histogram) does

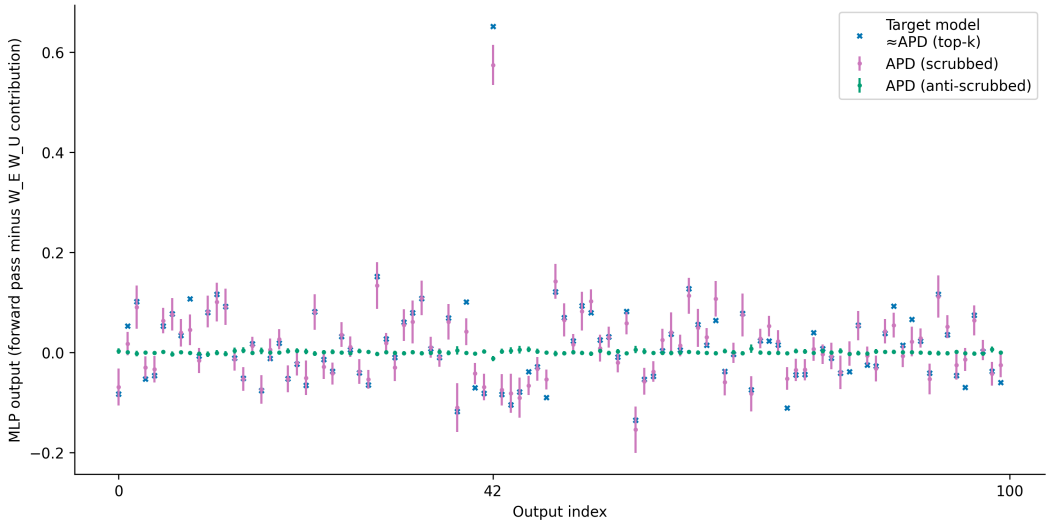

Figure 7: Output of multiple APD forward passes with one-hot input $x_{42} = 1$ over 10k samples, where half of the parameter components are ablated in each run. Purple lines show 'scrubbed' runs (parameter component corresponding to input index 42 is preserved), while green lines show 'anti-scrubbed' runs (component 42 is among those ablated). The target model output is shown in blue, which is almost identical to the output on the APD sparse forward pass (i.e. APD (top-k)). In this plot we only show the MLP output for clearer visualization. The embedding matrices are not decomposed and thus the residual stream contribution does not depend on APD components.

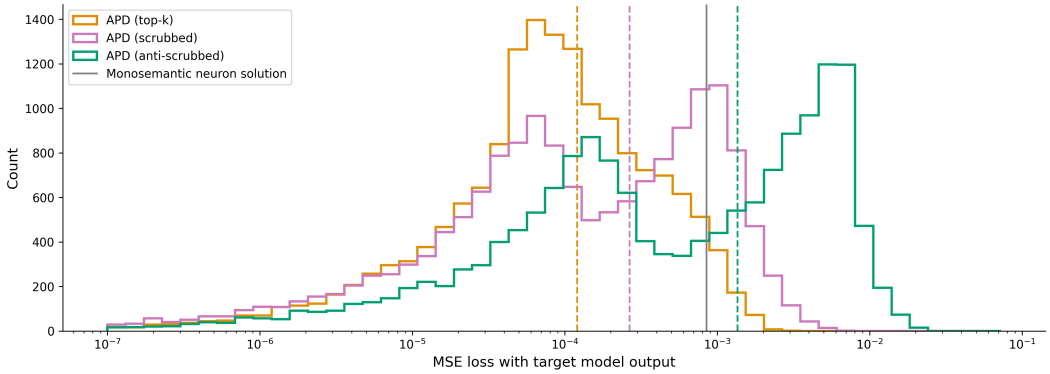

Figure 8: MSE losses of the APD model on the sparse forward pass ("top-k") and the APD model when ablating half (50) of its parameter components ("scrubbed" when none of the components responsible for the active inputs are ablated and "anti-scrubbed" when they are ablated). The gray line indicates the loss for a model that uses one monosemantic neuron per input feature. The dashed colored lines are the mean MSE losses for each type of run.

cause a large increase in MSE. This suggests that parameter components have mostly specialized to implement the computations for particular input features.

However, some parameter components appear to partially represent secondary input feature computations. This causes the visibly bimodal distributions of the scrubbed runs that can be seen in the figure: When these components are ablated, the loss of the model may be high when the secondary input feature is active. These components have the opposite effect on the loss when they are not ablated in the anti-scrubbed runs, making both scrubbed and anti-scrubbed losses bimodal. Preliminary work suggests that this can be improved with better hyperparameter settings or with adjustments to the training process, such as using alternative loss functions (Appendix A.2.3) or enforcing the APD components to be rank-1. We further quantify the extent to which some components partially represent secondary input feature computations in Appendix C.2.

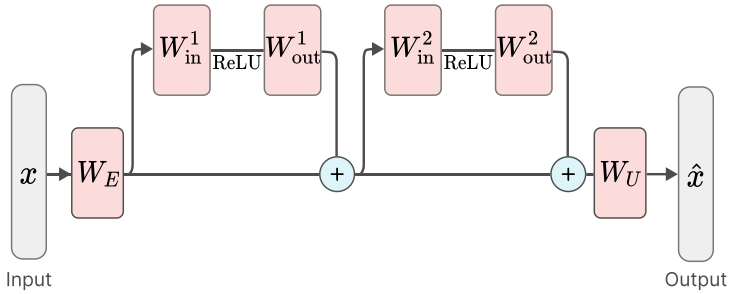

Figure 9: The architecture of our Toy model of Cross-Layer Distributed representations using a 2-layer residual MLP. We fix W_E to be a randomly generated matrix with unit norm rows, and $W_U = {W_E}^T$.

3.3 Toy Model of Cross-Layer Distributed Representations

We have seen how APD can learn parameter components that represent computations on individual input features, even when those computations involve neurons that contribute to the computations of multiple input features (Section 3.2). However, those computations take place in a single MLP layer. But realistic neural networks seem to exhibit cross-layer distributed representations [Yun et al., 2021, Lindsay et al., 2024]. In this section, we show how APD naturally generalizes to learn parameter components that represent computations that are distributed across multiple MLP layers[7].

We extend the model and task used in the previous section by adding an additional residual MLP layer (Figure 9). This model still performs compressed computation, but now with representations that are distributed across multiple layers. In this model, W_E is again a fixed, randomly generated embedding matrix with unit norm rows and $W_U = W_E^T$. We keep the residual stream width of $d_{\text{resid}} = 1000$ and 100 input features, but our 50 MLP neurons are now split across layers, with 25 in each of the two MLPs. We train APD on this model with 200 parameter components (allowing for 100 to die during training).

APD Results: Toy Model of Cross-Layer Distributed Representations

APD finds qualitatively similar results to the 1-layer toy model of compressed computation presented in Section 3.2. We see that the APD model learns parameter components that use neurons with large contribution values in both MLP layers (Figure 10, bottom). Again, we find that the computations occurring in each parameter component closely correspond to individual input feature computations in the target model (Figure 10, top versus bottom). For confirmation that the target model and APD model in the 2-layer distributed computation setting yield results that closely matching those observed in the 1-layer scenario, see Appendix C.3 and Appendix C.4, respectively.

However, the results exhibit a larger number of imperfections compared to the 1-layer case. In particular, more components represent two input feature computations rather than one. As in the 1-layer case, we again notice that batch top-k can cause some parameter components to not fully represent the computation of an input feature, and instead rely on activating multiple components for some input features (see Appendix C.4)[8].

4 Discussion

We propose APD, a method for directly decomposing neural network parameters into mechanistic components that are faithful, minimal, and simple. This takes a '*parameters-first*' approach to mechanistic interpretability. This contrasts with previous work that typically takes an '*activations-first*' approach, which decomposes networks into directions in activation space and then attempts to construct circuits (or 'mechanisms') using those directions as building blocks [Olah et al., 2020b, Cammarata et al., 2020, Cunningham et al., 2023, Bricken et al., 2023, Marks et al., 2024].

[7]For further validation that APD can identify cross-layer distributed representations, we apply it to a hand-coded network that implements a gated trigonometric function (Appendix B.1).

[8]Further analysis can be found in the "Toy Model of Cross-layer Distributed Representations (2 layers)" section of this WandB report.